

## Introduction

There are several APRS projects published on GitHub. This project differs from those in two ways. It provides functionality to form custom messages within the APRS standard. Second it can be used with a voltage-controlled oscillator (Si5351B) to directly create APRS messages. Typical APRS projects control a microphone input or require a specific FM radio chip. The main advantage of this design is that the same chip can generate a WSPR message for long distance status messages and APRS messages for significant amounts of data downloads.

This APRS software is designed to be used as a simple tracker but also can support experimenters wanting to collect data over a wide area using custom APRS messages. Several utility functions are provided to help in formatting custom messages that comply with the APRS standard. This allows the packets to contain data and other information such as images.

Currently the software has only been tested with the SAMD21 version, but none of the code is specific to the processor, so it is highly likely that it can be utilized with one of the other processors.

For balloon applications this project can be easily combined with WSPR\_ZERO2 to provide WSPR transmissions.

## APRS Packet

The APRS packet (message) has two parts the **header** and the **payload**. The software has helper functions to construct a properly formatted APRS message. A packet is sent with the `send_packet()` function. The first argument is the header type, and the second argument is the type of payload. For example,

```
send_packet(_BEACON, _POSITION);
```

Will produce the header "**K9YO-11**>BEACON,WIDE2-1"

and the body ",:!4317.00N/08750.63W"

where the longitude and latitude are properly formatted for a standard APRS position

This represents a basic message, but much more can be sent in a legally formatted APRS message.

## APRS Header

The APRS header is comprised of three fields.

1. The first is the sending callsign with a number extension e.g. W2XX-11. The extension is called the SiD. It can have one of only 16 values. In many cases, it indicate what type of vehicle or location the transmission originated from. This is the APRS convention for the 16 allowed values of the SiD;

- 0 - Home Station, Home Station running IGate.
- 1 - Digipeater, Home Station running a Relay Digi, Wx Digipeater
- 2 - Digipeater [#2 or] on 70CM
- 3 - Digipeater [#3]
- 4 - HF to VHF Gateway
- 5 - IGate (Not home station)
- 6 - is for Operations via Satellite
- 7 - Kenwood D7 HH
- 8 - is for boats, sailboats and ships (maybe 802.11 in the future)
- 9 - is for Mobiles
- 10 - is for operation via The internet only
- 11 - is for APRS touch-tone users (and the occasional **Balloons**)
- 12 - Portable Units such as Laptops, Camp Sites etc.
- 14 - is for Truckers
- 15 - is for HF

This convention is regularly violated by balloons and used to designate different balloons flying at the same time. Or designate different flights. However, in no case can a APRS signal from a transmitter send a value other than these 16. Higher numbers may be used when forwarding WSPR messages to the APRS network.

2. The second is the DEST field. Typically, this is determined by the equipment manufacturer of the transmitter. Each manufacturer has a designated dest field. There are some special dest field indicators that are not related to a manufacturer that are available for experimental purposes. The code utilizes three different dest indicators which are described in the next section.
3. The third field indicates how the packet should be forwarded. For balloons, this should be WIDE2-1. This minimizes the amount of unneeded repeater activations that a high-altitude balloon could cause.

### APRS Dest Field

The dest field is used for various purposes. It can be used to send a message to a particular callsign. More often it indicates the type of equipment that has sent the message. This program can use APIGA to simulate the message protocol coming from a qrplabs APRSLite tracker. This allows additional tracking information to be automatically uploaded to SoundHub. See the Time, Location and Status body discussed in the next section.

The dest field can also be used to indicate that the message body is an experimental body that does not follow any of the conventions for a standard APRS body. The examples below use APX783 as the dest for an experimental message. The APX indicates it is experimental and the number is arbitrary.

The first argument to the send\_packet() function determines the format of the header. It can create these three types of packet headers.

- `_NORMAL` will produce this header "K9YO-11>APX783,WIDE2-1" The first field is the callsign with an SID, the second field indicates that this is an experimental package by using APX. The value 783 is arbitrary. It can be any 3 alphanumeric characters.
- `_BEACON` will produce this header "**K9YO-11**>BEACON,WIDE2-1"

- `_BALLOON` will produce this header "**K9YO-11**>APLIGA,WIDE2-1" The second field, APLIGA is the designator for the qrplabs APRSLite tracker. The use of this header will allow the packet to be automatically picked up and plotted by SoundHub.

## APRS Payload (Body)

The program is capable of sending 5 different types of APRS bodies. Each has different formatting constraints. Helper functions are provided to simplify the required formatting. Each of the packet headers can be combined with one of packet bodies to form a message with the desired information.

### `_STATUS`

This will send a string of alpha numeric characters. The status message length when used without position information can be up to 255 characters. Status can contain any printable characters. Such as "Arduino Tracker by K9YO."

Helper functions: `APRSSetStatus(char* status)`

### `_POSITION`

This body will contain the latitude and longitude in the APRS format e.g. "!4317.00N/08750.63W"

Helper function: `APRSLatLong(float latitude, float longitude)`. This function will convert the variables Longitude and Latitude in decimal degrees to the necessary information for the proper transmission format for the `_POSITION` argument.

### `_POSITION_STATUS`

This body will contain the latitude and longitude in the APRS format followed by a status message of up to 43 characters e.g. "!4317.00N/08750.63W\ Arduino Tracker by K9YO." Use the helper functions for `_STATUS` and `_POSITION` when requesting this body to be sent

### `_DATA`

This produces an experimental packet of up to 253 characters of data. There is utility functions provided to add float, int and strings to build a comma delimited body of data. This type of packet can be forwarded to your iGate. Here is an example:

```
{B,1572,EN62ag,3.14,1,
```

The first two characters will be added automatically by the helper function. The B is a user defined character to identify the following data in the body. This allows the tracker to send several different packets of this type that can be combined by the destination. Image data is sent using this technique.

Helper Functions `APRSDatInit('B')`, `APRSDataAppendInt(int)`, `APRSDataAppendFloat(float)`, `APRSDataAppendChars(char*)`

### `_BALLOON`

This payload should be combined with the `_BALLOON` header. It contains data in a particular format that will be automatically picked up by SoundHub and plotted along with the path of the balloon. Here is an example:

```
@083251h4217.00N/08757.83W0009/025/A=002000 -5TxC -5.23C 250.22hPa 6S 2024-2-4
```

The above body contains: the character "@" the time (hhmmss), latitude (degrees, minutes ddmm.mm), longitude (degrees, minutes), the letter "O", course (degrees clockwise from North), speed (knots), altitude (feet), CPU temperature (C), air temperature (C), pressure (hPa), number of satellites.

Helper Function: `APRSSensorInfoInit()`; and `APRSSetSensorInfo` e.g.

```
APRSSetSensorInfo(-5, -5.24, 250.23, 6, (int)year(), (int)month(), (int)day());
```

-5 – CPU temperature

-5.24 – air temperature

250.23 – pressure in hPa

6 – number of satellites

## Geofence

The code contains logic to prevent transmission over countries where not allowed. If `GEOFENCE_position(latitude, longitude)` returns false, transmission is not allowed.

## Putting it all together

In the file `APRSLoop.h` are two functions that show examples of how to use the helper functions to construct the messages. `APRSBegin()` calls the helper functions to set the necessary variables. `SendAPRSPackte()(int msg)` calls `APRSBegin()` then transmits the appropriate message type.

## Generating the RF

The APRS signal is a 1200 baud signal at 144.39 MHz. This program is written to use an SI5351B 20-QFN (Note: The 16-QFN package uses a different register set that is incompatible with JTEncode). Option B provides a voltage controlled oscillator controlled by the voltage on pin 3. This voltage is changed by a digital output from the cpu to generate the tones required by APRS.